

Text Preprocessing

Turning raw, messy text into clean input a model can actually learn from

Why Preprocess Text?

Garbage in, garbage out — raw text is too messy for models to use directly

Raw Text Looks Like This:

*"OMG!! This movie was sooo GOOD... 10/10 would recommend :) Check it out here: <http://bit.ly/xyz123>
 #mustwatch"*

Problems:

- Mixed casing ("GOOD" vs "good")
- Punctuation & emoticons carrying noise
- URLs and HTML artifacts
- Slang, hashtags, abbreviations

Why It Matters:

- **Reduces vocabulary size**
Fewer unique tokens → less sparsity, faster training
- **Removes irrelevant noise**
Models learn signal, not formatting artifacts
- **Improves generalization**
"Run", "running", "Ran" are treated consistently
- **Required by downstream steps**
Vectorization (BoW, TF-IDF, embeddings) needs clean tokens

Structured vs. Unstructured Text

Preprocessing needs are fundamentally different for each

Structured Text

Organized in fixed fields, schemas, or tables — e.g., form entries, database records, transaction logs, structured JSON/CSV.

Typical preprocessing focus:

- Standardizing formats (dates, currency, units)
- Handling missing / null values
- Encoding categorical fields
- Field-level validation, not full tokenization

Unstructured Text

Free-form, natural language — e.g., reviews, tweets, articles, transcripts, emails, chat logs.

Typical preprocessing focus:

- Full pipeline: cleaning, tokenization, normalization
- Stop-word removal, stemming/lemmatization
- Handling slang, emojis, spelling errors
- Sentence/discourse-level structure matters

It Depends: Problem Statement & Data Type

There is no single “correct” pipeline — the task and the data shape every decision

Problem / Data Type	Preprocessing Strategy
Sentiment analysis (reviews)	Keep negations & punctuation like "!"; handle emojis; light stemming
Search / document indexing	Aggressive lowercasing, stopword removal, and stemming for recall
Machine translation	Minimal preprocessing — preserve word order, case, and punctuation
Structured logs / form data	Field parsing & format standardization, not full tokenization
Social media text	Handle hashtags, mentions, slang, emojis, and heavy misspellings
Legal / medical documents	Preserve domain terms exactly; avoid aggressive stemming

The Complete Preprocessing Roadmap

Every step covered in detail on the following slides

1 Text Cleaning (HTML/URL/Email Removal)

3 Case Normalization

5 Handling Emojis & Emoticons

7 Tokenization

9 Stemming

11 Handling Numbers & Dates

2 Removing Special Characters & Punctuation

4 Expanding Contractions

6 Spelling Correction

8 Stop-Word Removal

10 Lemmatization

12 POS Tagging & Named Entity Recognition

1

Text Cleaning — Noise Removal

Strip away structural artifacts that carry no linguistic meaning

BEFORE

```
"<p>Check this out!! Visit http://bit.ly/xyz123 or email us at info@site.com</p>"
```

AFTER

```
"Check this out Visit or email us at"
```

How It's Done

- Regular expressions to match and remove URLs, emails
- HTML/XML parsers (e.g., BeautifulSoup) to strip tags
- Remove non-printable / control characters
- Collapse repeated whitespace and line breaks

When to Use It

Essential for web-scraped data, emails, and HTML-sourced content.
Usually the very first step in the pipeline.

Caution

Over-aggressive cleaning can delete meaningful content embedded in markup (e.g., table data).

2

Removing Special Characters & Punctuation

Reduce symbol noise while preserving meaningful structure

BEFORE

"Wow!!! This is amazing??? #bestday @friend"

AFTER

"Wow This is amazing bestday friend"

How It's Done

- Regex character-class filtering (e.g., keep only [a-zA-Z0-9])
- String translation tables for fast bulk removal
- Selective removal — keep hyphens in compound words
- Preserve currency symbols/decimals for numeric fields

When to Use It

Useful for topic classification and search indexing where symbols add no discriminative value.

Caution

For sentiment analysis, "!" and "?" carry emotional signal — removing them can hurt accuracy.

3

Case Normalization (Lowercasing)

Make casing consistent so identical words aren't treated as different tokens

BEFORE

"Apple launched a new iPhone. I love APPLE products!"

AFTER

"apple launched a new iphone. i love apple products!"

How It's Done

- Simple `.lower()` / `.casefold()` transformation
- Applied after cleaning, before tokenization
- Locale-aware casefolding for non-English text

When to Use It

Standard for most classification, clustering, and search tasks to shrink vocabulary size.

Caution

For Named Entity Recognition, capitalization is a key signal ("Apple" the company vs. "apple" the fruit) — skip this step.

4

Expanding Contractions

Convert shortened forms into their full grammatical form

BEFORE

"I don't think it's working, y'know?"

AFTER

"I do not think it is working, you know?"

How It's Done

- Dictionary-based mapping (don't → do not, can't → cannot)
- Handles informal contractions (y'know, gonna, wanna)
- Applied before tokenization to avoid split apostrophes

When to Use It

Improves consistency for downstream tokenization and reduces vocabulary fragmentation.

Caution

Contraction dictionaries are language- and dialect-specific; informal/slang contractions need custom mappings.

5

Handling Emojis & Emoticons

Emojis carry strong signal in informal text — decide to convert or remove

BEFORE

"This movie was amazing! 😂🔥 loved it :)"

AFTER

"This movie was amazing! laughing fire loved it happy"

How It's Done

- Emoji-to-text libraries (e.g., convert 😂 → "face_with_tears_of_joy")
- Emoticon dictionaries (":)" → "happy", ":(" → "sad")
- Alternative: strip entirely for formal/structured text

When to Use It

Critical for social media sentiment analysis — emojis often carry as much signal as words.

Caution

Irrelevant (and should simply be stripped) for legal, medical, or structured/tabular text.

6

Spelling Correction

Fix typos and non-standard spelling before they fragment the vocabulary

BEFORE

"I recieved the pakage yesterday, tahnk you!"

AFTER

"I received the package yesterday, thank you!"

How It's Done

- Edit-distance based correction (e.g., SymSpell, pyspellchecker)
- Statistical/contextual correction (e.g., TextBlob, neural spell-checkers)
- Domain-specific dictionaries to avoid false corrections

When to Use It

Important for noisy, user-generated text — reviews, chat logs, social media, support tickets.

Caution

Can incorrectly “correct” domain jargon, proper nouns, or intentional stylistic spelling.

7

Tokenization

Split text into the fundamental units that all later steps operate on

BEFORE

"I love NLP. It's fascinating!"

AFTER

[I, love, NLP, ., It, 's, fascinating, !]

How It's Done

- Word tokenization — splits into individual words
- Sentence tokenization — splits into sentences
- Subword tokenization (BPE, WordPiece) — used by transformer models

When to Use It

The foundational structural step — nearly every later step (stopwords, stemming, POS) operates on tokens.

Caution

Naive whitespace splitting breaks on “don’t”, “New York”, “\$5.00” — use a proper tokenizer library.

8

Stop-Word Removal

Filter out very common words that carry little topical meaning

BEFORE

[the, movie, that, i, watched, was, really, good]

AFTER

[movie, watched, really, good]

How It's Done

- Remove tokens from a predefined stop-word list (the, is, at, of...)
- Custom stop-word lists tuned to the domain/corpus
- Frequency-based removal of very high-frequency corpus terms

When to Use It

Common in search indexing and topic/document classification to reduce noise and dimensionality.

Caution

Removing “not”, “no”, “never” can flip meaning entirely in sentiment analysis — use a task-aware list.

Crude, rule-based suffix stripping to a common root form

BEFORE

[studies, studying, caring, happiness]

AFTER

[studi, studi, car, happi]

How It's Done

- Porter Stemmer — the classic, widely used rule-based algorithm
- Applies suffix-stripping rules (e.g., “SSES” → “SS”, “IES” → “I”)
- Snowball Stemmer — an improved, multi-language successor

When to Use It

Fast and lightweight — good for search/indexing where speed matters more than linguistic accuracy.

Caution

Output is often not a real word (“studi”, “happi”) — unsuitable when readability of tokens matters.

10

Lemmatization

Dictionary- and grammar-aware reduction to the true root word

BEFORE

[studies, studying, caring, better]

AFTER

[study, study, care, good]

How It's Done

- Uses a lexical database (e.g., WordNet) plus POS tags
- Verb, noun, and adjective forms are resolved differently
- Slower than stemming but linguistically accurate

When to Use It

Preferred when output must be a valid word — e.g., feature interpretation, topic labeling, chatbots.

Caution

Requires accurate POS tagging first; mismatched POS can produce an incorrect lemma.

11

Handling Numbers & Dates

Decide whether to keep, remove, or standardize numeric information

BEFORE

"Order #4521 was placed on 5/6/2024 for \$129.99"

AFTER

"Order <NUM> was placed on <DATE> for <CURRENCY>"

How It's Done

- Replace with placeholder tokens (<NUM>, <DATE>, <CURRENCY>)
- Normalize date formats to a single standard (ISO 8601)
- Remove entirely if numbers are irrelevant to the task

When to Use It

Critical for structured/financial/medical text where numeric consistency matters for downstream parsing.

Caution

Blind removal loses information vital to tasks like invoice parsing or medical dosage extraction.

POS Tagging & Named Entity Recognition

Enrichment steps that add grammatical and semantic structure

BEFORE

"Barack Obama was born in Hawaii."

AFTER

POS: [NNP, NNP, VBD, VBN, IN, NNP] NER: [Barack Obama→PERSON, Hawaii→GPE]

How It's Done

- POS tagging assigns grammatical categories (noun, verb, adjective...)
- NER identifies real-world entities (people, places, organizations, dates)
- Both typically run on tokenized, case-preserved text

When to Use It

Feeds lemmatization accuracy and supports information extraction, question answering, and search.

Caution

Requires case and punctuation to be largely intact — run before aggressive lowercasing/cleaning.

Key Takeaways

- Text preprocessing exists to turn noisy, inconsistent text into clean, model-ready input
- Structured text needs format standardization; unstructured text needs the full linguistic pipeline
- The right steps depend entirely on the problem statement and the nature of the data
- Cleaning, case, tokenization, stopwords, and stemming/lemmatization form the core sequence
- Enrichment steps like POS tagging and NER add structure for information-extraction tasks
- Always ask: does this step help or hurt the specific signal my task depends on?

Next: Text Representation — Bag of Words, TF-IDF, and N-grams

Assignment 2: Data Pre-processing for NLP

The objective of this assignment is to understand, implement, compare, and analyze preprocessing techniques on **two different datasets** belonging to different domains, problem statements, and languages.

Assignment Requirements

Select **two datasets** satisfying the following conditions:

- Different **problem statements**
- Different **application domains**
- Different **languages** (for text datasets)

Examples:

Dataset 1

English Sentiment Analysis

Medical Diagnosis

Spam Detection

English News Classification

Dataset 2

Hindi Fake News Detection

Financial Fraud Detection

Product Recommendation

Hinglish Hate Speech Detection

